Lakehouse Platform for Lending, Insurance and Recharge

Design Document — Data Engineer Take-Home Exercise

Aman Bhardwaj

1. The problem, as I read it

Three businesses — lending, insurance and recharge — each grew their own OLTP systems, event streams, partner files, third-party APIs and hand-maintained spreadsheets. That is a normal way for a company to arrive at the stated symptom: **nobody can say with confidence whether a given table is current or correct.** That sentence is the brief; everything below exists to make it false.

I read it as two separable and separately hard questions:

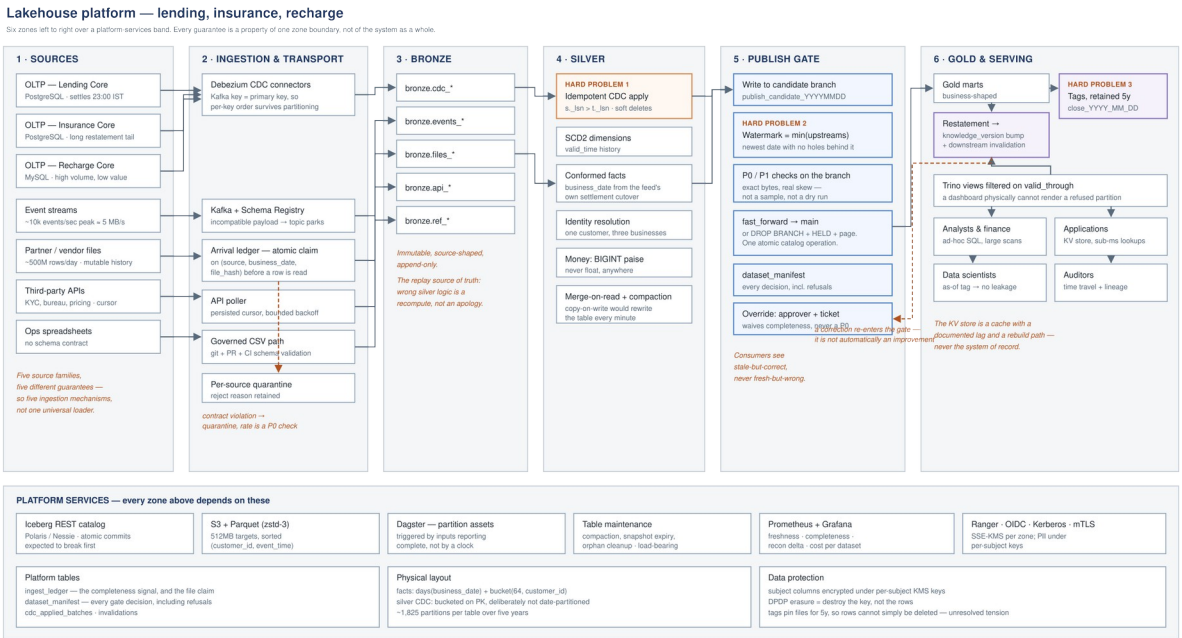
- **Correct** — the transformation is right. Same raw fact in, same number out, deterministically.
- **Current** — the platform knows what it does *not* yet know, and says so, rather than publishing a plausible number built on an incomplete input.

> **A design that solves correctness but not currency produces wrong answers with total confidence — worse than the status quo, which at least makes people suspicious.**

Constraints taken as given: ~10k events/sec peak, ~500M rows/day from files, 50M customers, five years of history, money as integer paise. **None of the hard problems is about volume** — 10k events/sec is roughly 5 MB/s. They are trust problems that happen to occur at a size where you cannot fix them by hand.

2. Architecture: what I chose, what I rejected, what it costs

Six zones left to right — sources, ingestion and transport, bronze, silver, publish gate, gold and serving — over a platform-services band every zone depends on. **The flow is the design:** every guarantee the platform makes is a property of one zone boundary, not of the system as a whole.



End-to-end platform. Source: docs/architecture/platform-overview.svg

Each row below is a one-line summary. Where the choice was genuinely contested, the alternatives and consequences in full are a record in **docs/adr/**; the last column says which, or points at the section here that carries it.

Layer	Chosen	Rejected	Cost of the choice	ADR
Table format	Iceberg on S3/Parquet	Delta (weaker branch/tag semantics for write-audit-publish); Hudi (larger operational surface)	Thinner SQL surface; write amplification managed by hand	0001
Catalog	Iceberg REST (Polaris / Nessie)	Hive Metastore (no multi-table txn, no branching); Glue (pins one cloud)	A newer component on every read and write path	0001
Processing	Spark — batch + Structured Streaming	Flink — better streaming engine, but nothing needs sub-second freshness	~30–60s latency floor; commit pressure on the catalog	0002
Orchestration	Dagster, partition-aware assets	Airflow — its unit is a task at a time of day, not “has date D arrived”	Less familiar; a learning curve for Airflow-fluent engineers	0002
Query / serving	Trino for analytics; KV store for apps	One API over one table — would serve all four consumers badly	Two serving paths to keep consistent, with a documented lag	§8
Compression	Zstandard level 3, dictionary on	Snappy (worse ratio, and ratio dominates over five years); gzip (slow reads); LZ4 (optimises write CPU, not scarce here)	Higher ingest CPU; excludes readers without zstd	0007
Encryption	SSE-KMS per zone; mTLS; per-subject keys for PII	Row deletion on erasure (collides with tag pinning); masking only (plaintext still at rest)	KMS on the PII read path; **crypto-shredding as erasure is not settled law**	0007
CDC write mode	Merge-on-read; copy-on-write for marts	Copy-on-write everywhere (the default) — rewrites every file holding a changed row, every minute	Read amplification between compactions; compaction becomes load-bearing	0006

2.1 Layer roles

- **Bronze** is source-shaped, append-only and immutable, and never gets smarter transformations. Its only job is to be a faithful replayable copy of what arrived and when, so that wrong silver logic six months from now is a recompute rather than an apology.
- **Silver** is conformed: CDC becomes ordered upserts, dimensions pick up SCD2 history, facts get a business_date from the feed's own settlement cutover, customers resolve to one identity across the three businesses, and every monetary column is a BIGINT of paise.
- **Gold** is business-shaped marts, and **nothing reaches it except through the publish gate.**

3. Getting data in

Each source family gets the mechanism suited to how it actually behaves and — more importantly — an explicit statement of what happens when it misbehaves.

Source	How it lands	When it misbehaves
OLTP cores (×3)	Debezium → Kafka, keyed by primary key so per-key order survives partitioning; applied by the idempotent applier (§6.1)	Replayed offsets are made inert by the LSN guard. An incompatible schema change parks the topic rather than writing garbage. A long outage is a replay from bronze, not a re-snapshot.
Event streams (~10k/sec)	Straight onto Kafka with a producer-assigned event_id; deduped over a 24-hour window, bucketed by **event time, not arrival time**	Duplicates are assumed, not hoped against. Events later than the window become a correction rather than a new arrival (§3.1).
Partner files (~500M rows/day)	Claimed atomically on (source, business*date, file*hash) before a row is read; **content hash, never filename**, decides new vs duplicate vs correction (§6.3)	Redelivery under a new name is a no-op; new content for a known date is a restatement, not an append. Malformed rows quarantine with a reason, and the quarantine rate is itself a P0 check.
Third-party APIs	Poller with a persisted cursor that does not advance past an unconfirmed page	Bounded backoff on 429/5xx. A vendor silently changing pagination is the failure I cannot detect cheaply, so row counts per poll are recorded and a discontinuity raises.
Ops spreadsheets	Deliberately no magic pipeline — a governed CSV path: git-tracked, PR-reviewed, schema-validated in CI	The defect in a hand-maintained spreadsheet is not the file format, it is the absence of review before a number reaches production. Automating the load without the review just automates the mistake.

3.1 Late-arriving events

“Late” means three different things, and conflating them is how revenue moves between reporting days:

Kind	Cause	Treatment
Late within the stream	A mobile client buffers offline and flushes hours later; a producer retries after a partition.	Bucketed by **event time** into its own business*date, deduped on event*id within a 24-hour window.
Beyond the dedupe window	An event arrives more than 24 hours after it occurred.	Written to a late-arrival partition and treated as **a correction to a published number** — it marks the business_date stale and re-runs the gate for exactly that date (§6.2, §6.3).
Late because we were down	Connector outage, then catch-up.	Not lateness. Kafka retained the events; the applier replays them in order and the LSN guard makes any

		overlap inert (§6.1).

> **A very late financial event is never dropped and never mixed into today — it is a restatement of the day it belongs to.**

Window policy is **per feed, not global** — recharge tolerates a short one, insurance needs a long one, and the window is a declared property of the feed rather than a tuning knob.

4. Physical design, capacity and cost

Several claims elsewhere depend on the physical layout, so it is stated rather than assumed.

4.1 Layout

- **Partitioning** — facts and events by days(business*date); five years is ~1,825 partitions per table, which keeps manifest counts sane while letting the gate and a restatement address exactly one day. Very large facts add bucket(64, customer*id) so customer joins are shuffle-light. **Silver CDC tables are bucketed on the primary key and not date-partitioned at all** — rows would migrate partitions on update and rewrite files for no benefit.
- **Clustering** — sort order (customer*id, event*time) within a partition, 512MB target files, compaction to that size within an hour of a partition being published. Small files are the tax micro-batching charges, and 1,440 commits/table/day generates them relentlessly.
- **Write mode** — merge-on-read on CDC targets, copy-on-write on marts. The applier owns the property set in code and asserts it at startup, so a target that silently reverted to the default fails loudly instead of quietly costing many times as much (ADR-0006).

4.2 Compression

Zstandard level 3, set as a table property so it belongs to the table rather than to whichever job wrote it. Dictionary encoding stays on and does most of the real work on the columns that dominate these tables — status, source*system, business*line, currency codes — because no general-purpose codec beats a dictionary on low-cardinality strings.

```
write.parquet.compression-codec = zstd
write.parquet.compression-level  = 3
```

Level 3 rather than the library default of 1, and well below the maximum: the ratio curve flattens sharply after ~3 while CPU keeps climbing, so levels in the teens buy a few percent of storage for a large share of ingest CPU on a path that runs every minute. Compression is not a space/time trade-off on object storage — it is cheaper on both stored bytes and scanned bytes. *These are published codec characteristics, not a benchmark against this data; column cardinality drives real-world ratio more than codec choice does.*

4.3 Encryption and key management

Three requirements that get conflated if they are not named apart:

1. **At rest** — SSE-KMS with a customer-managed key **per zone**, so a compromise or revocation is scoped to a layer and key usage is auditable. Server-side, because Spark, Trino and compaction all read the same files.
2. **In transit** — TLS everywhere, mTLS between platform services; Kerberos for Hadoop-protocol access, OIDC for humans, Ranger for authorisation.
3. **Erasure** — the hard one. §6.3 pins files for five years, and **you cannot delete rows a tag is holding.** So PII columns are encrypted under a per-subject key wrapped by a KMS master key, and an erasure request **destroys the key, not the rows**. Tagged reads still resolve — files, row counts and aggregates stay exactly as reported — but the personal columns return ciphertext nobody can decrypt.

Crypto-shredding as erasure is a reasonable and widely-argued position, not settled law. It is the constraint most likely to force a redesign, and I would want legal in the room.

4.4 Capacity and cost

Quantity	Value	Derivation
Event throughput	~5 MB/s at peak	10,000/s × ~500 bytes
File volume	~25–40 GB/day compressed	500M rows × ~250 B ÷ 3–5 for Parquet+zstd
Five-year storage	~60–75 TB	12–15 TB/year × 5, plus a single-digit % for tag-pinned restatements
Rows retained	~900 billion	500M × 365 × 5
Catalog commits	~288,000/day ≈ 3.3/sec	200 tables × 1,440 minutes

Storage is not the interesting number; compute is. Streaming needs a standing cluster sized for peak, not average. File loads and mart rebuilds burst and can take spot capacity. Compaction is a third standing draw that people forget to budget and that competes with ingestion for the same pool. I would instrument **cost per table** from day one — not because the total is alarming, but because the distribution is always more skewed than anyone expects, and that is where the savings are.

5. The unifying idea: five clocks

What made three problems feel like one design: every row carries five distinct notions of time, and almost every subtle bug in a system like this comes from collapsing two of them.

Clock	Meaning
event_time	When the fact occurred.
ingest_time	When the platform first saw it.
business_date	Which operational day it settles into — not the calendar day it arrived.
valid_time	The window during which a stated fact was considered true.
knowledge_time	**What the platform believed as of a given moment.** The axis a restatement moves along, and the axis an auditor's question is actually phrased in.

> **An Iceberg table's snapshot history already IS a knowledge-time axis. I do not need to build bitemporal versioning — I need to stop discarding it and name the moments that matter.**

That is what §6.3 does, and it is why leakage-free training data for data scientists falls out of the same mechanism instead of needing a second one.

6. Three hard problems

I chose these over other candidates — schema evolution across three OLTP sources, cross-source identity resolution, spreadsheet governance — because they sit on the critical path and because **getting any one of them wrong produces a failure that is invisible in production**: the table stays queryable, nothing pages, and the wrongness surfaces months later when someone reconciles against a source. That is a much worse failure shape than a job that crashes, and it is the shape the brief's pain statement describes. Code in src/dataplatform/; the reasoning that belongs next to a line of code lives in its docstring rather than being repeated here.

6.1 Applying CDC to Iceberg safely under replay

Why it's hard. The obvious implementation reads a micro-batch, MERGEs it, then commits offsets — two commits against two systems with no transaction spanning them. If the process dies between them the batch is re-applied, and whether that is harmless depends entirely on whether the MERGE is idempotent. A “last row wins” MERGE is not, because the replay can interleave differently with a later batch that has since landed for the same key.

Diagram of CDC apply under replay: docs/architecture/hardproblem1.png

Where the guarantee lives. Ordering is carried by the source database's own log position as *lsn* — *not source.ts*ms, which is not monotonic across a failover and can tie within one transaction — and the MERGE gates its UPDATE on it:

```
MERGE INTO target AS t USING staged AS s
ON t.pk = s.pk
WHEN MATCHED AND s._lsn > t._lsn THEN UPDATE SET <guarded columns>
WHEN NOT MATCHED THEN INSERT (...) VALUES (...)
```

> **That single predicate is both the ordering and the idempotency guarantee. A replay carries old positions, so it matches no UPDATE and inserts nothing — whether it is one second later or ten batches later.**

- **Deletes are soft**, so a late stale insert for a deleted key is rejected by the same comparison instead of resurrecting it. Consumers read a view that filters tombstones.
- **Columns the source did not send are not written.** Postgres omits an unchanged TOASTed column from the WAL record and Debezium substitutes a sentinel *string*, which written verbatim looks like data. The collapse carries the latest real value forward **per column** and the MERGE guards each column with a changed-flag; a column genuinely set to NULL still writes NULL.
- **The per-batch marker is an optimisation, not the guarantee** — it skips a MERGE the LSN guard would have made inert anyway. Marker *after* the data, never before: a crash in between costs one redundant MERGE, where the reverse order would skip a batch that never landed and lose data.

Cost accepted: one partition-pruned lookup per micro-batch, on 7-day retention. Deliberately not a probe of the target on a *batch*id column — a sha256 defeats min/max file pruning, degrading that probe to a scan of a 50M-row table once a minute. Full record: **ADR-0003**.

6.2 Deciding a dataset is complete, and refusing to publish when it isn't

Why it's hard. Completeness is not observable. You cannot know that no more data is coming for a business date, only whether everything you were promised has arrived. And the resulting failure is quiet in a specific way:

> **A mart joins a healthy loan feed to a stalled repayment feed. The output is queryable, non-empty, internally consistent — and wrong: loans genuinely appear to have no repayments. No null check fires. No referential check fires.**

Diagram of the publish gate: docs/architecture/hardproblem2.png

Solution. Completeness is **declared** per source in an arrival contract and decided against, never inferred from the data. Publication is write-audit-publish: the candidate goes to a per-business-date branch no consumer reads, graded P0/P1 checks run against those exact bytes, and only a clean run is fast-forwarded onto main in one atomic catalog call. A failure drops the branch and records HELD; main keeps serving the last good state.

Two ways to get the watermark wrong, not one:

4. A maximum **across** upstreams ignores the laggard — the obvious bug. The effective watermark is the *minimum*, and a missing signal collapses the result to unknown rather than being read optimistically.
5. A maximum **within** one upstream ignores holes: a source that reported D-3 and D but never D-2 says D, and the gate waves through a close built on a missing day. So the rule is **the newest date with an unbroken run behind it**. *This is the subtler of the two, and the one that reads as if it were already correct.*

Cost accepted: a stalled upstream holds the whole dependent tree past its freshness SLA and pages someone. That is intended — a late number is recoverable, a wrong number that has been reported on is not — so the escape hatch is designed rather than stubbed. An **Override** carries an approver, a ticket and a reason, publishes as OVERRIDDEN rather than PUBLISHED, and **can waive completeness but never a P0.** Incompleteness is a judgement call; a negative money column is not. Full record: **ADR-0004**.

6.3 Absorbing a correction without lying about what we used to believe

Why it's hard. The first two problems are about getting data in correctly; this one is about **having been wrong in public.** A settlement file for the 3rd arrives on the 7th with different totals, finance closed the month on the 5th using the old numbers, and an auditor later asks why the closed report and the live table disagree. Both answers have to remain available, and the difference has to be explainable.

Diagram of restatement and knowledge time: docs/architecture/hardproblem3.png

Solution. Every delivery is classified against the ingest ledger by **content hash** — NEW, DUPLICATE (a no-op) or RESTATEMENT. The lookup deliberately considers claimed-but-not-yet-complete rows: restricting it to complete rows classifies a redelivery arriving mid-load as brand new, and loads the same

file twice. A restatement claims the next knowledge*version by MERGE; Iceberg gives conflict* detection*, not automatic re-execution, so the retry loop is load-bearing, and the winner is established by reading its own worker*id back — "my MERGE did not raise" is not the same statement as "I own this claim".

- The correction replaces the partition in **one Iceberg commit**, detaching old files as it attaches new ones, so no reader sees it empty or doubled.
- It writes to the candidate branch, so **a correction failing its own P0 checks is held like any other** — a correction is not automatically an improvement.
- Tag and invalidation fire from the publish path, never the write. Both are claims about *published* state; emitting at write time tells marts to rebuild from a partition the gate might still hold.

Ordinary snapshots expire after 7 days; a tag is retained for five years, so "as reported on 31 March" becomes an ordinary SQL read an auditor runs themselves:

```
SELECT * FROM table VERSION AS OF 'close_2026_03_31'
```

Cost accepted, and genuinely unresolved. Tags pin data files, so superseded versions stay on disk for five years — a single-digit percentage of storage, and a bargain against not being able to answer the auditor. The sharper cost is the collision with a DPDP erasure request (§4.3). Full record: **ADR-0005**.

7. Data quality and alerting

Two failures have to be told apart, because the right response differs: **the data is wrong** (publishing it causes decisions on a false number) versus **the data is late or unusual** (it may be correct and merely odd). Treating them the same produces one of two outcomes, both bad: block on everything and the gate becomes something people route around; block on nothing and the gate is decoration.

Severity	Meaning	Effect	Examples
P0	The data is wrong	**Blocks publication. No override can waive it.**	negative principal_paise; duplicate primary key; reconciliation delta against the source's own control totals
P1	The data is unusual	Publishes, raises a ticket, tracked	row count outside its trailing band; quarantine rate above threshold; freshness inside SLA but degrading

Checks count violations, not booleans — whoever is paged at 03:00 needs the blast radius first, and "1 row" versus "4.2M rows" are very different pages. **Four signals per dataset:** freshness against the arrival contract; completeness (watermark vs business date); **reconciliation delta against the source's own control totals** — the only check that catches a transformation that is internally consistent and still wrong; and cost per dataset, where a spike is often the first visible symptom of a join that started fanning out.

Escalation follows consequence, not the label. A P0 hold with a downstream finance dependency pages; elsewhere it raises a business-hours ticket, because the guarantee held and nobody is reading

wrong data. **Override rate per dataset is itself a reported metric** — a routine override is a disabled gate with extra paperwork, and that degradation is organisational rather than technical.

A side-car DQ framework cannot hold the blocking role. Great Expectations or Deequ are good at expressing and reporting checks, and either would be a reasonable way to author the P0/P1 suites. What neither can do is sit *inside* the publish transaction: run out-of-band they observe data consumers can already read, which detects a bad partition instead of preventing one. So the checks run against the candidate branch before the fast-forward, and the gate — not the framework — owns the decision.

8. Serving: what each consumer actually gets

Consumer	Gets	Why not something else
Analysts & finance	Gold marts through Trino, views filtered on the manifest's valid_through	The filter is in the serving path, not a runbook — a dashboard **physically cannot** render a partition the gate refused.
Applications	Sub-millisecond lookups from a KV store fed off conformed silver	A point lookup behind a query planner is tens of ms at best with no SLA under concurrency. A cache with a documented lag and a rebuild path, **never the system of record.**
Data scientists	Reads as of the Iceberg tag current at their training cutoff	Makes label leakage from a later correction **structurally impossible** rather than a code-review concern. Same mechanism as the auditor's — no second system.
Auditors	Time travel, lineage, and the manifest of every gate decision including refusals	"We refused this partition, for these reasons, at this time" is the evidence actually asked for six months later.

9. Living with it: run, fail, recover, and where it breaks first

Day to day. Dagster materialises partition-scoped assets when their inputs report complete, not at a time of day. Prometheus and Grafana watch the four signals in §7. A held partition pages or tickets by consequence; a P1 raises a ticket. Compaction, snapshot expiry and orphan cleanup run as a standing maintenance job — which is what keeps §6.3's audit history affordable rather than an unbounded liability.

Recovery has three shapes:

6. A **failed job** is a retry, and every path here is built to be re-runnable.
7. **Bad transformation logic** is a bronze replay — recompute silver and gold for the affected dates. This is why bronze stays immutable and why five years of it earns its storage.
8. **Bad published data** is the restatement path — the only one needing a human decision rather than an operator, because someone has to say whether finance re-closes.

Target: any single business date recomputable within one business day — a claim about the shape of the pipeline, not a benchmarked number.

9.1 Where it breaks first

Rank	Failure	Why, and what is honest about the claim
1	**CDC write amplification**	Iceberg defaults to copy-on-write; at a 1-minute trigger against a 50M-row table with updates scattered across the key space, that rewrites a large fraction of the table every minute. Mitigated by ADR-0006, but the mitigation only holds if compaction keeps pace. *On the arithmetic this is the strongest candidate for the first thing here to buckle under load.*
2	**Metadata accumulation in the catalog**	~1,440 snapshots/table/day, ~288,000 platform-wide. At 3.3 commits/sec that is **not** a throughput problem and it would be wrong to claim it is — the concern is accumulation: ~10,000 live snapshots per table under 7-day expiry, with commits to one table serialising. *Arithmetic, not a benchmark; the 200-table figure is invented.*
3	**Maintenance falling behind during a burst**	Festival traffic spiking recharge, with compaction competing against the ingestion it is cleaning up after. The backlog degrades reads slowly enough that nobody notices until queries are already slow.
4	**The gate's blast radius** — the one I'd watch soonest	One chronically-late upstream can hold a large dependent subtree. Not a technical failure: **it is that people start asking for overrides routinely, and a routine override is a disabled gate with extra paperwork.**

10. Honesty section

10.1 What I deliberately left out

Left out	Statement
Cross-business identity resolution	Mechanism designed, matching logic not written. Its own hard problem; I judged the three I picked sharper and more central to the stated pain.
Insurance and recharge cutover times	I state 23:00 IST for lending and payments because the given facts imply that flow. I **do not have** the other two and did not invent them — §3 differentiates them on behaviour I can reason about instead.
Schema evolution	The registry rejects incompatible payloads and the topic parks. That is a policy, not a design; additive change

	should flow through automatically and does not.
Alerting and override approval plumbing	Named seams. The Override is designed and tested; **who may sign one** is an organisational decision I lack the context to make.
The erasure-versus-audit resolution	§4.3 is a direction, not a design: key-per-subject at 50M customers, the KMS request rate on the read path, and a tagged read whose key is gone mid-query are all unworked.
Per-source business calendar	The hole-detection rule assumes every calendar date is owed, so a source idle at weekends over-holds. Named seam, unimplemented.

10.2 Where I'm not fully sure

- **The 6-hour due_after_close default** — confident in the shape (declare it, don't infer it), but I have not load-tested whether six hours is realistic at this volume, and it is almost certainly too generous for recharge and too tight for insurance.
- **Whether write amplification really beats the catalog** as the first bottleneck. Both claims are arithmetic. Measuring actual rewrite volume per micro-batch is the first thing I would do with production data.
- **The contiguity rule** assumes every calendar date is owed. For a genuinely intermittent source that is wrong and over-holds.
- **Overwrite atomicity under concurrent readers** — §6.3 asserts an overwrite commit is never observed half-applied. That is documented Iceberg behaviour, and it is also the property I would be least surprised to find broken under real concurrent load, and the one I have not tested.
- **The top-level entry points** — run*stream, ingest*delivery and publish_restatement are the three composition points and none is exercised by a test. The guarantees underneath each are; the wiring between them is not.

10.3 Three traps in this design, and where AI was used

Each of the three hard problems has an implementation that reads as if it were already correct and is not. Naming them is more useful than restating what the design gets right, because these are the places a reviewer should push:

9. **The per-batch marker is not the idempotency guarantee, and must not be relied on as one.** It is mutable, and a probe keyed on it is not metadata-pruned — a sha256 defeats min/max pruning, so that lookup scans. The LSN predicate carries correctness; the marker is an optimisation, and §6.1's tests prove correctness with it removed.
10. **source_watermark must not take a maximum within an upstream.** A hole in a source's history would pass the gate — the exact failure the gate exists to prevent, hiding inside the function that implements it. Hence the unbroken-run rule.
11. **A TOASTed column must never overwrite a real value with Debezium's sentinel string.** Collapsing to the whole max-_lsn row and merging with SET * writes a column the source never sent as if it were data — silent, unpaged, and wrong until someone reads that column. Hence per-column carry-forward plus a changed-flag guard in the MERGE, covered by three tests.

AI assistance. Used to draft and iterate the code, tests and documents against a design I had already decided on, to render the diagrams from a layout I specified box by box, and as an adversarial reviewer of my own drafts. Every API it produced I checked against the Iceberg documentation rather than trusting, and that is what caught a REPLACE WHERE clause (Delta syntax Iceberg does not have) plus three system.* procedure calls for branches and tags that do not exist — those are DDL. Exactly the errors a fluent-sounding draft produces and a reviewer who reads code as code will catch.

The decisions are mine: the choice of these three problems, the min-not-max and no-holes-behind-it watermark rules, missing-as-unknown, the LSN predicate as the guarantee with the marker demoted, content-hash classification, tags-as-knowledge-time, the override-waives-completeness-never-a-P0 line, and the five-clocks framing.

10.4 Code size

src/dataplatform/ is roughly 1,200 lines across four modules, of which about 600 are executable once docstrings and comments are stripped, against a suggested 200–400. **It is over, and I would rather say so than derive a number that makes it look compliant.** The overage is almost entirely prose, and deliberate: each module header states the guarantees it owns, and the reasoning for a decision sits next to the line implementing it. The brief says it reads code as code, because that is where a guarantee either holds or does not. Strip the prose and the mechanisms are compact; the prose is there because the mechanisms are the part worth defending.